

System Calls

System Calls

- Provide the Interface between a process and the OS

System Calls

WHEN?

- System calls are usually made when a process in user mode requires access to a resource.
- Then it requests the kernel to provide the resource via a system call.

System Calls

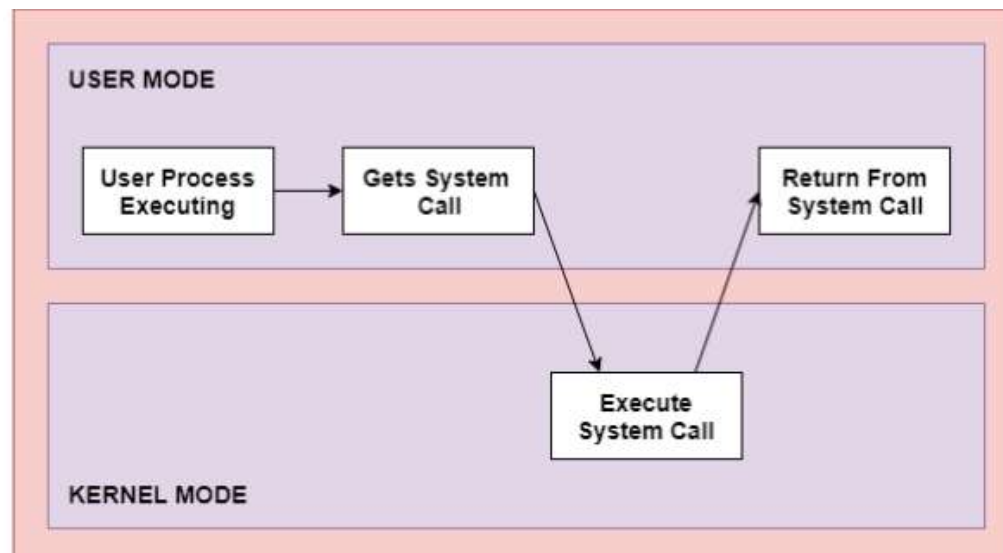
WHEN?

System calls are required in the following situations –

- 1) If a file system requires the creation or deletion of files.
- 2) Reading and writing from files also require a system call.
- 3) Creation and management of new processes.
- 4) Network connections also require system calls. This includes sending and receiving packets.
- 5) Access to a hardware devices such as a printer, scanner etc. requires a system call.

System Calls

- 1) The processes execute normally in the user mode until a system call interrupts this.
- 2) Then the system call is executed on a priority basis in the kernel mode.
- 3) After the execution of the system call, the control returns to the user mode
- 4) Execution of user processes can be resumed.



System Calls

- **Generally available as Assembly language instructions.**
- **Certain systems allow system calls to be made directly from a higher level language program,**
 - In this case, they resemble predefined functions or subroutine calls

System Calls

- Languages like **C,C++, Perl** have been defined **to replace Assembly language** for system programming.
 - They **allow system calls to be made directly from programs.**
- **Unix system call may be invoked directly from C or C++ program.**

System Calls

- **Mostly accessed by programs via a high-level [Application Programming Interface \(API\)](#) rather than direct system call use**
- Three most common APIs are
 - Win32 API for Windows,
 - POSIX API for POSIX-based systems (including virtually all versions of UNIX, Linux, and Mac OS X), and
 - Java API for the Java virtual machine (JVM)

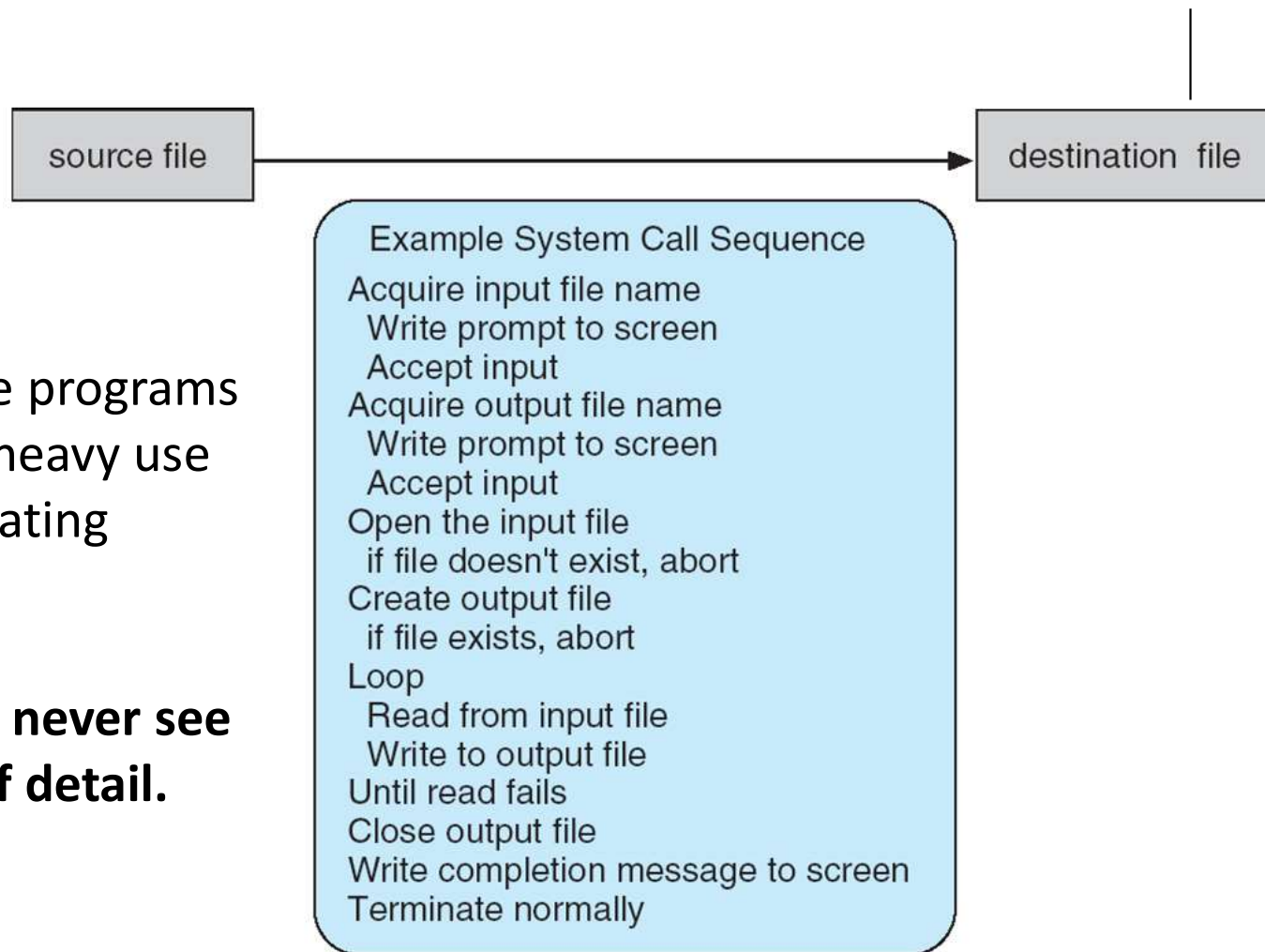
Example of System Calls

- System call sequence to copy the contents of one file to another file



Example of System Calls

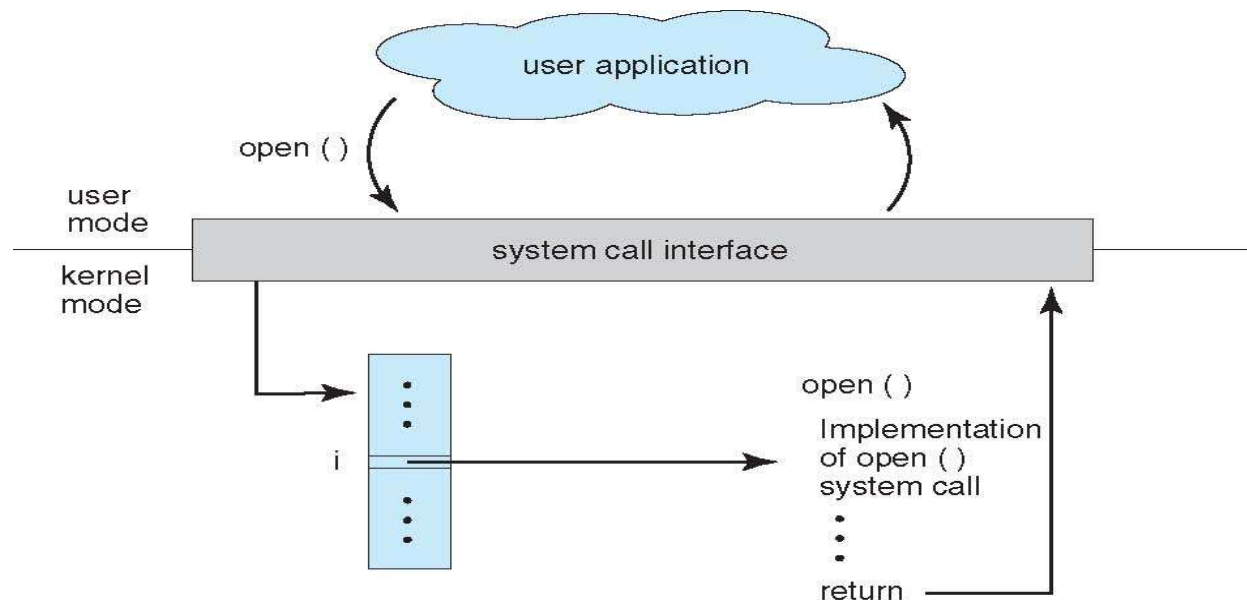
- System call sequence to copy the contents of one file to another file



- Even simple programs may make heavy use of the operating system.
- Most users never see this level of detail.**

System Call Implementation

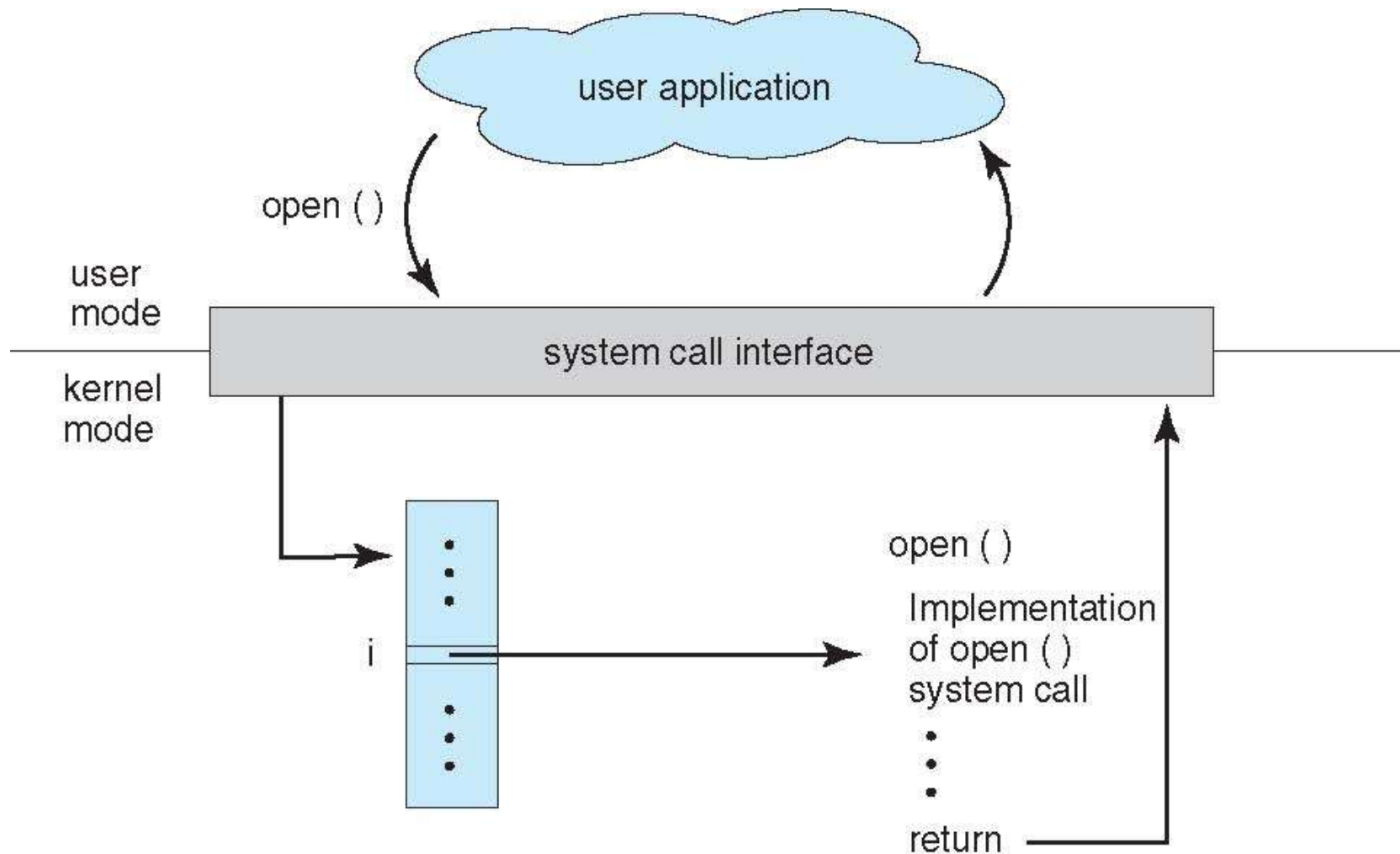
- Typically, a number is associated with each system call
 - **System-call interface** maintains a table indexed according to these numbers
- The system call interface
 - **invokes the intended system call in OS kernel and**
 - **returns status of the system call and any return values**



System Call Implementation

- **The caller need know nothing about how the system call is implemented**
 - Just needs to obey API and **understand what OS will do as a result of call**
 - **Most details of OS interface hidden from programmer by API**
 - Managed by run-time support library (set of functions built into libraries included with compiler)

API – System Call – OS Relationship



System Call Parameter Passing

- System Calls occur in different ways
- Often, more information is required than simply identity of desired system call
 - **Exact type and amount of information vary according to OS and call**
 - Eg- To get input,
 - **we need to specify file or device to use as the source and**
 - the address and **length of the memory buffer** into which the input should be read.

Example of Standard API

EXAMPLE OF STANDARD API

As an example of a standard API, consider the `read()` function that is available in UNIX and Linux systems. The API for this function is obtained from the man page by invoking the command

```
man read
```

on the command line. A description of this API appears below:

```
#include <unistd.h>

ssize_t  read(int fd, void *buf, size_t count)
```

return value	function name	parameters
-----------------	------------------	------------

A program that uses the `read()` function must include the `unistd.h` header file, as this file defines the `ssize_t` and `size_t` data types (among other things). The parameters passed to `read()` are as follows:

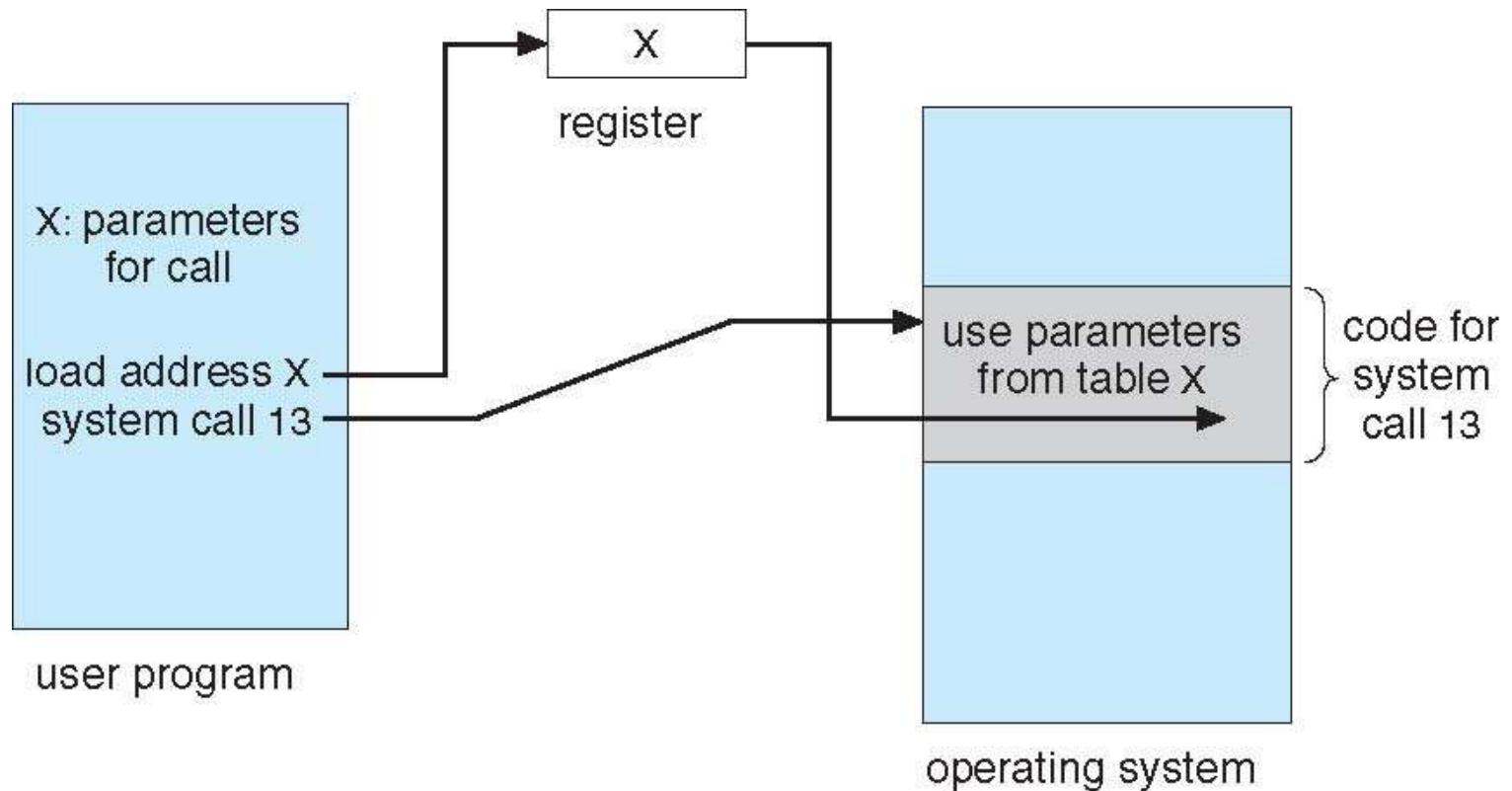
- `int fd`—the file descriptor to be read
- `void *buf`—a buffer where the data will be read into
- `size_t count`—the maximum number of bytes to be read into the buffer

On a successful read, the number of bytes read is returned. A return value of 0 indicates end of file. If an error occurs, `read()` returns `-1`.

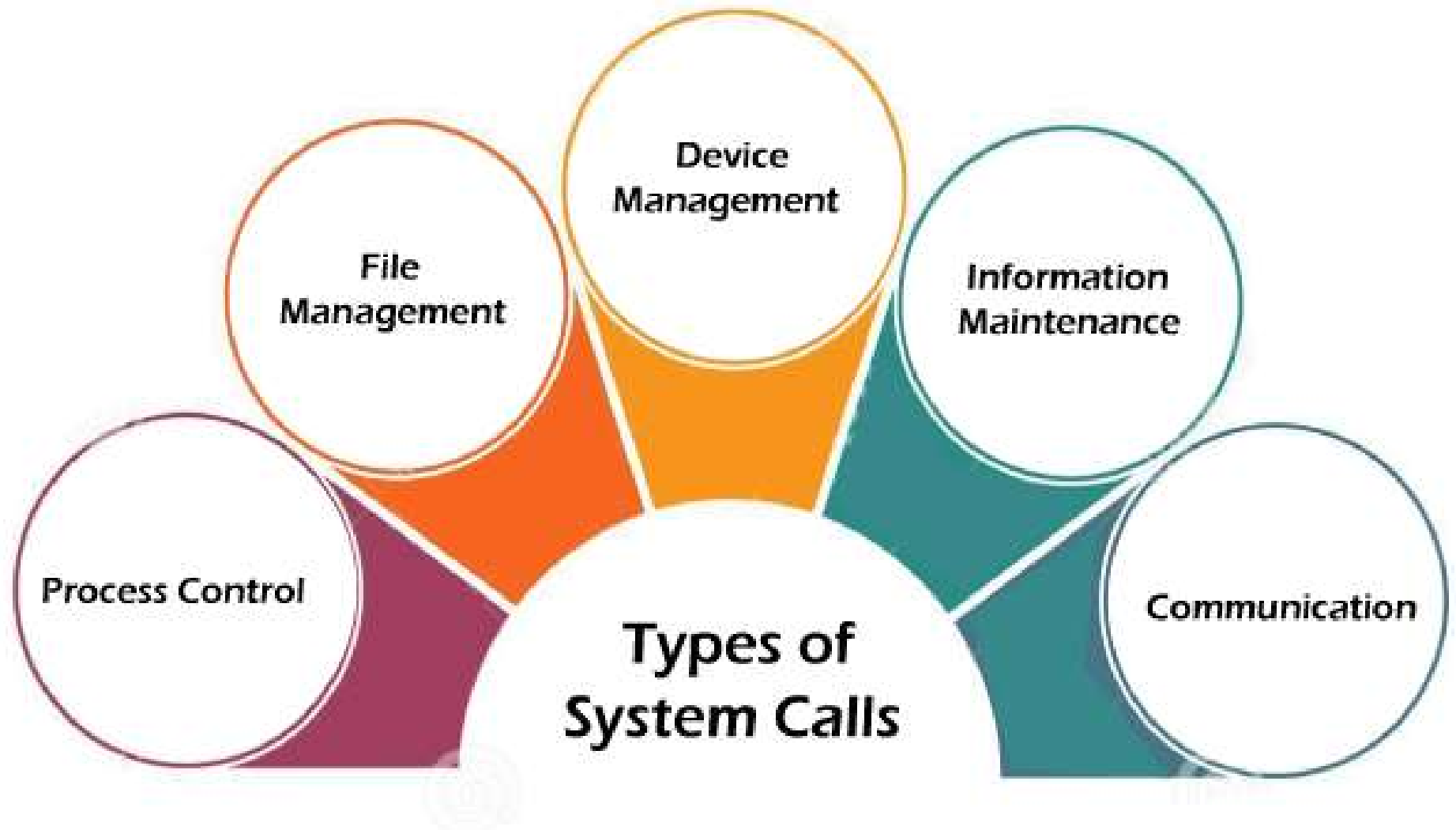
System Call Parameter Passing

- Three general methods used to pass parameters to the OS
 - Simplest: **pass the parameters in registers**
 - In some cases, may be more parameters than registers
 - **Parameters stored in a block, or table, in memory, and address of block passed as a parameter in a register**
 - This approach taken by Linux and Solaris
 - **Parameters placed, or pushed, onto the stack by the program and popped off the stack by the operating system**
- Block and stack methods do not limit the number or length of parameters being passed

Parameter Passing via Table



Types of System Calls



Types of System Calls

- Process control
 - **create process, terminate process**
 - **end, abort**
 - load, execute
 - **get process attributes, set process attributes**
 - **wait for time**
 - wait event, signal event
 - allocate and free memory
 - Dump memory if error
 - **Debugger** for determining **bugs, single step** execution
 - **Locks** for managing access to shared data between processes

Types of System Calls

- File management
 - **create file, delete file**
 - **open, close file**
 - **read, write, reposition**
 - **get and set file attributes**

Types of System Calls

- Device management
 - **request device, release device**
 - **read, write, reposition**
 - **get device attributes, set device attributes**
 - **logically attach or detach devices**

Types of System Calls (Cont.)

- Information maintenance
 - **get time or date, set time or date**
 - get system data, set system data
 - **get and set process (process id),** file, or device attributes

Types of System Calls (Cont.)

- Communications
 - create, delete communication connection
 - send, receive messages
 - If **message passing model** to **host name** or **process name**
 - From **client** to **server**
 - **Shared-memory model** create and gain access to **memory regions**
 - transfer status information
 - attach and detach remote devices

Types of System Calls (Cont.)

- Protection
 - **Control access to resources**
 - **Get and set permissions**
 - Allow and deny user access

Examples of Windows and Unix System Calls

	Windows	Unix
Process Control	CreateProcess() ExitProcess() WaitForSingleObject()	fork() exit() wait()
File Manipulation	CreateFile() ReadFile() WriteFile() CloseHandle()	open() read() write() close()
Device Manipulation	SetConsoleMode() ReadConsole() WriteConsole()	ioctl() read() write()
Information Maintenance	GetCurrentProcessID() SetTimer() Sleep()	getpid() alarm() sleep()
Communication	CreatePipe() CreateFileMapping() MapViewOfFile()	pipe() shmget() mmap()
Protection	SetFileSecurity() InitializeSecurityDescriptor() SetSecurityDescriptorGroup()	chmod() umask() chown()